初探：从0开始的AI-Agent开发踩坑实录

文想 大淘宝技术 2025年10月15日 03:19 浙江 53人



本文主要阐述作者通过亲身实践，探索利用AI Agent实现开源应用Helm Chart自动化生成的实践历程。

01 引言

这事儿得回到两周前，彼时我刚入职，正兴致勃勃地想接下来会做些什么，周会上说到新需求的进展遇到了一个小问题——面临上百个开源应用的k8s部署适配，TL表示现在啊，AI提效是关键，“咱们能不能搞个AI？”“就给它一个开源应用，它一下自己部署全了。”

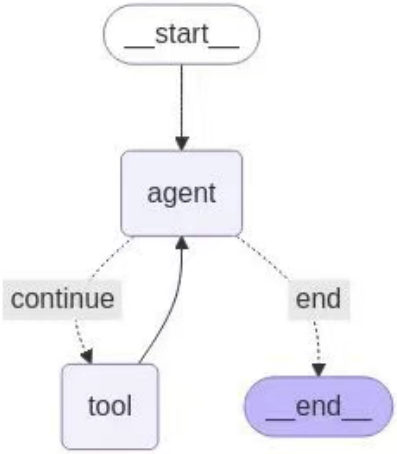
在那个瞬间，我脑海里闪过的一个宏大的图景——一个全能的应用部署agent，好吧冷静下来，还是聚焦于一个小的开始，k8s部署物的自动生成。（输入一个开源项目（如 GitHub 链接），AI Agent 能够分析项目，并输出一个可直接部署的 Helm Chart 包）。

我一头扎进了 AI Agent 的世界。这个过程，比我想象的要曲折。经历了初期“AI 无所不能”的天真幻想，也体验了Agent 陷入无限循环时的茫然。最终，磕绊地实现了一个初级的最小可行性方案。现在把这段尚在进行时的探索之旅记录下来，作为个人学习的总结，也希望能和大家交流，得到前辈们的指点。

02 怎么做一个agent

定义 Agent 的角色

万事开头难，尤其是做一个没有接触的东西。最初的我理解的agent模样：



只要提供一堆工具和prompt，agent就可以自行探索–分析–决策–执行直到解决任务。尽管从宏观角度上来说这样并没有错，但这却让我踩进了第一个坑——**让LLM控制一切：给LLM一堆工具让它自由发挥企图它能够按照心里的目标实现，而让LLM控制一切的结果就是，不完善的prompt和tools带来了无限循环。**

LangChain给了一个宏观指导，它把构建 Agent 的过程分成了几个阶段，从定义到部署。

LangChain

Stage	Define	Design Standard Operating Procedure	Build MVP for Core Prompts	Connect & Orchestrate	Test & Iterate	Deploy, Scale & Refine
Description	*Identify a realistic agent use case and gather 5-10 concrete scenarios the agent should handle*	*Create a step-by-step procedure, breaking down the task(s) into clear steps a human would follow*	*Design agent architecture; Build prompt(s) that handle the core reasoning*	*Connect to live data sources and build orchestration logic*	*Evaluate agent against examples; Identify failures & iterate to improve reliability & performance*	*Deploy agent & monitor how users interact; continuously refine based on real usage patterns.*
Duration	~1-2 week(s)	~1-2 week(s)	~1-2 week(s)	~2-3 week(s)	~1-2 month(s)	Ongoing
Output	Clear agent task scope & concrete use cases	Clear Standard Operating Procedure of step-by-step workflow, incorporating identified scenarios	Agent architecture and core prompt(s) that works with static data	End-to-end agent with live data	Agent with systematically tested performance	Improved agent with expanded capabilities
Stakeholders	Product Owner, Subject Matter Expert	Product Manager, Subject Matter Expert	Product Manager, AI & Prompt Engineer	Engineers	Product Manager, QA Engineer	PM, Engineers, Support
Example: Building an Email Agent	Example tasks: • Categorize email by priority • Schedules meetings • Answer product questions	Step-by-step procedure: • With incoming emails, label with email category & response priority • Checks calendar availability & schedule meeting • Draft response & queue for human review	Email agent architecture design with core prompts, incl. email routing & response generation; Test with manually provided context and sample data	Example integrations: • Gmail & Outlook for email access • Google & Outlook Calendar • CRM database for sender context	Evaluate email responses over success criteria: response quality, accuracy, and professional tone	Monitor traffic, use cases, and feedback. Iterate to improve performance and add additional features

(图源：https://blog.langchain.com/how-to-build-an-agent/)

回头看，我前期的混乱，本质上就是在 **Stage 1: Define** 阶段的无所适从。我一心想快速实现**Stage 3: Build MVP**，却忽略了最根本的问题：**在这个任务里，AI 的边界到底在哪里？**

我应该让 AI 做什么？又应该让工程代码做什么？

这个问题，应该是 AI 应用探索的起点。前期我东一榔头西一榔头，一会儿尝试让 AI 自己写 shell 脚本去扫描目录，一会儿又让它去决策先分析哪个文件，结果就是 AI 的表现极其不稳定，输出的结果惨不忍睹。

在踩了坑之后，我才明白，Define 阶段要回答的，不仅仅是“做什么功能”，更是“AI 以何种方式参与工作”。

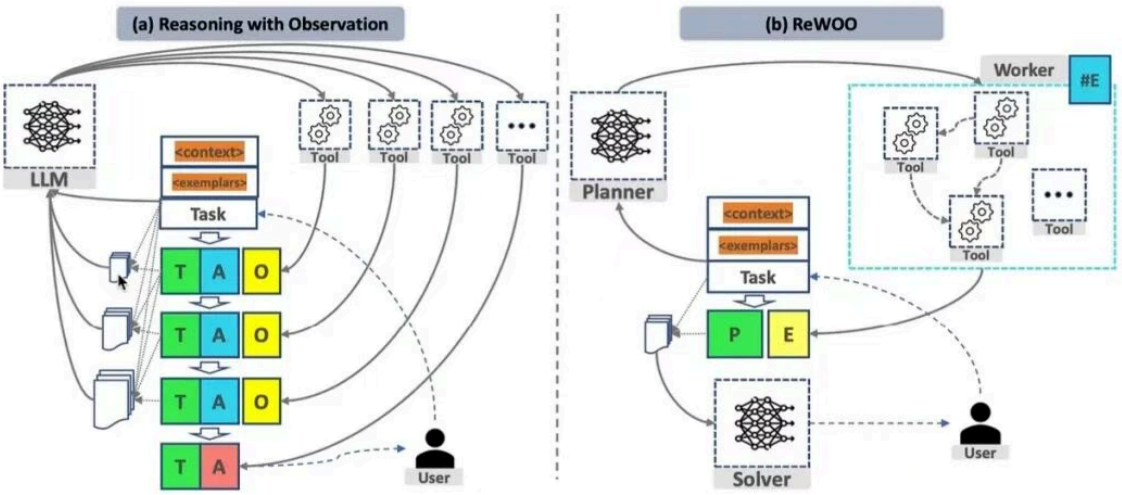
- AI 能力的边界

尽管从理想情况来说，一切人能够清晰衡量和验证的事物都能被AI解决（源自Jason wei），但应用落地还是要受限于当下的模型。有可能随着模型的发展，一句话的prompt就能让ai实现当下多agent完成的任务。个人实践来看，AI 是在做分析者（Analyst） 与 决策者（Decision-Maker）。

- 作为分析者：这是 AI 最容易胜任的角色。AI 的核心任务是感知和理解。它不直接执行任何变更操作，阅读庞大的信息迅速抓住信息的重点。例如，它可以阅读持续监控 Prometheus 指标和 Pod 日志，聚合所有相关信息——包括异常指标、错误日志、相关的变更事件，然后生成一份根本原因分析报告。尽管依然有“幻觉”的出现，但现在的AI在这个方面已经做得不错了。
- 作为决策者：让 AI 做决策，意味着它需要对环境有深刻的理解，甚至具备一定程度的“常识”，在已知的模型能力下，往往和高质量的prompt和上下文强相关。要让 AI 胜任这个角色，必须给它提供一套明确的行动框架：清晰的工具集、详尽的工具使用场景、固定的工作流，甚至要细化到每个决策节点的触发时机。 比如，不能简单地告诉 AI “更新服务”，而是要将这个任务拆解成一个它能理解的工作流。在这种情况下，执行越小越清晰任务的agent往往比执行复杂任务的agent表现得更好，也更容易控制。

- Agent的行为范式

业界已经有很多关于 Agent 行为范式的探讨：
ReAct（Reasoning and Acting）《ReAct: Synergizing Reasoning and Acting in Language Models》：https://arxiv.org/pdf/2210.03629



（图源：ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models）

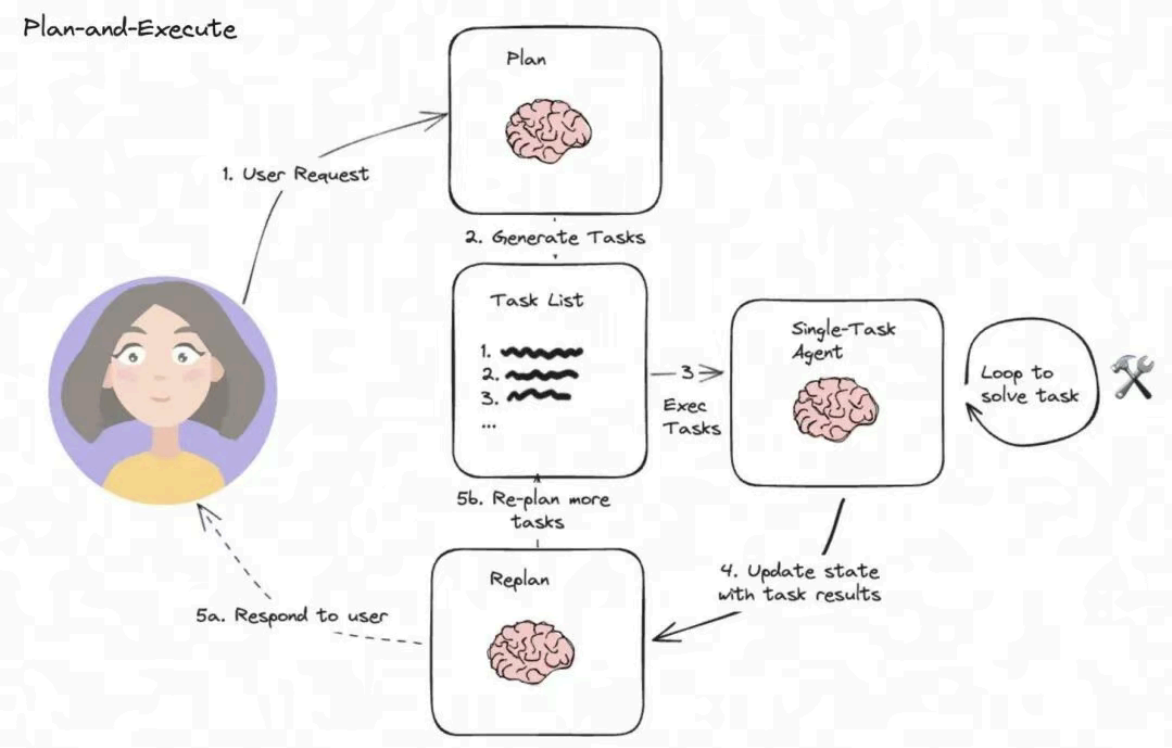
ReAct 的核心是 Thought -> Action -> Observation 的循环 。LLM 思考一下，决定调用一个工具（Action），然后观察工具返回的结果，再进行下一轮思考 。该框架本质上创建了一个反馈循环，每次完成此循环时（即每次代理采取行动并根据该行动的结果进行观察时），代理必须决定是否重复或结束循环。LangGraph 中的 ReAct 代理模块通过预定义的系统提示实现，与该模块一起使用的 LLM 不需要任何其他示例即可充当 ReAct 代理。

```
请尽力回答以下问题。您可以使用以下工具：


维基百科：维基百科的封装器。当您需要回答关于人物、地点、公司、事实、历史事件或其他主题
duckduckgo_search：DuckDuckGo 搜索的包装器。当你需要回答有关时事的问题时非常有用
计算器：当您需要回答有关数学的问题时很有用。
使用以下格式：
问题：您必须回答的输入问题
思考：你应该时刻思考自己该做什么
行动：要采取的行动，应该是 [Wikipedia、duckduckgo_search、Calculator] 之一
动作输入：动作的输入
观察：行动的结果
...（这个想法/行动/动作输入/观察可以重复N次）
想法：我现在知道最终答案了
最终答案：原始输入问题的最终答案
开始！
问题：{输入}

想法：{agent_scratchpad}
```

plan-and-execute



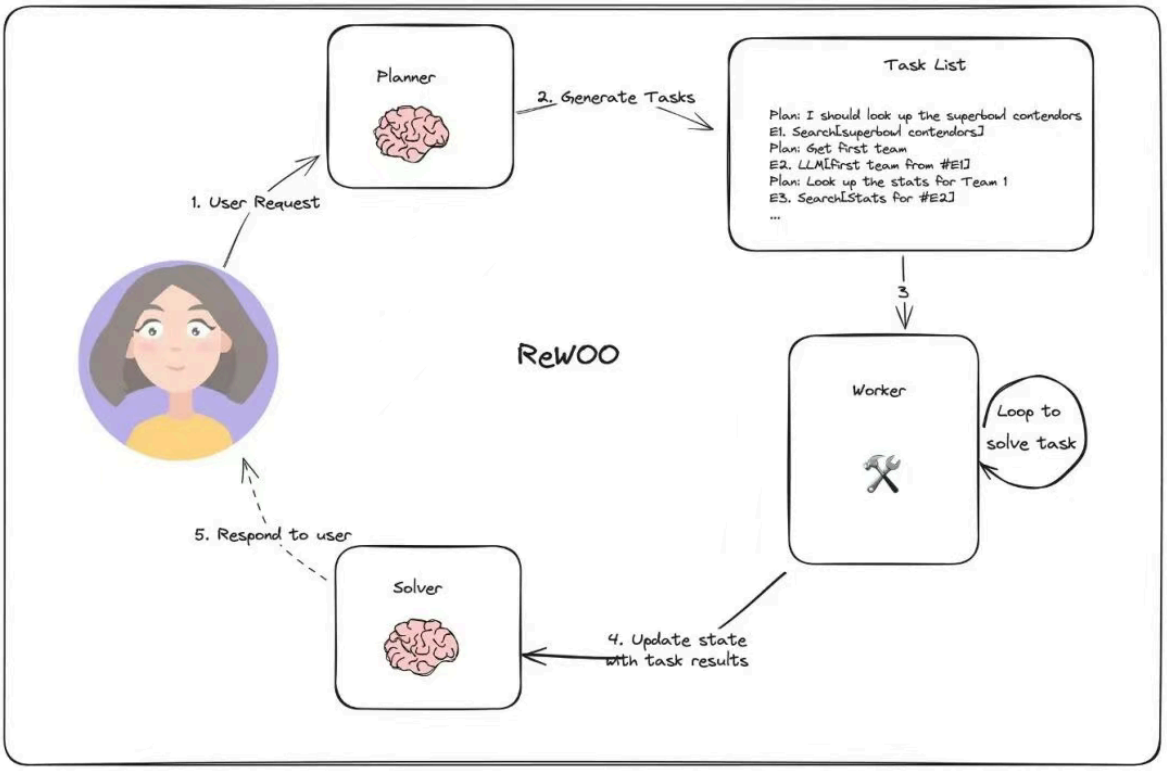
（图源：https://github.com/langchain-ai/langgraph/blob/main/docs/docs/tutorials/plan-and-execute/plan-and-execute.ipynb）

plan-and-execute旨在克服ReAct类代理的局限性，通过在执行任务前明确规划所有必要步骤。这种方法旨在提高效率、降低成本并提升整体性能。其核心工作流程包含三个阶段：

- 规划阶段 ： 接收用户输入，并生成一个用于完成大型任务的多步骤计划或任务清单；
- 执行阶段 ： 接收计划中的步骤，并调用一个或多个工具来按顺序完成每个子任务；
- 重规划阶段：根据执行结果动态调整计划或返回；

manus的Agent像是借鉴了这种思路，首先生成任务清单，再对着清单逐个执行。

ReWOO（ Reasoning WithOut Observation ）



（图源：https://github.com/langchain-ai/langgraph/blob/main/docs/docs/tutorials/rewoo/rewoo.ipynb）

ReWOO是一种创新的增强语言模型（Augmented Language Model, ALM）框架。它将复杂任务的解决过程分解为三个独立的模块：

- 规划器（Planner）： 基于 LLM 推理，提前生成任务蓝图（步骤顺序与逻辑），无需工具实时反馈。
- 执行器（Worker）： 按蓝图并行调用外部工具（如搜索、计算器），收集证据。
- 求解器（Solver）： 综合分析蓝图和执行证据，生成最终答案（含纠错总结）。

ReWOO 最显著的特点是拥有一个独立的 Solver 模块。它专门负责整合规划意图与工具执行结果，屏蔽 Worker 的执行细节，直接基于蓝图和证据生成最终答案；在执行阶段（Worker 步骤），ReWOO 不查看外部工具返回的中间结果细节。

与 Plan and Execute 关键区别：

- ReWOO Worker 只负责执行工具调用，无需 LLM 驱动。
- ReWOO 没有重新规划步骤（蓝图固定）。

Agent框架

Agent框架我没有进行过多的研究，还没有一个像springboot对java web那样绝对统治地位的框架出现，完全可以自行实现一个简易的Agent框架，构建LLM + Tools + Workflow的组合。通过编写一些“胶水代码”，手动管理模型的输入输出、调用外部工具（如API或数据库），并控制整个任务的流程。当然使用一个成熟的框架能够更快地聚焦于任务业务部分。社

区中已经涌现出许多优秀的Agent框架，我使用的是LangChain和LangGraph，当前阶段还不需要维护非常复杂的逻辑。

prompt工程

我最初也曾低估了Prompt工程的复杂性，认为它不过是“提需求”的简单环节。但当一个初步可行的框架和流水线搭建起来后，所有后续的质量优化、效果对齐和稳定性问题，几乎都聚焦到了与Prompt的持续“搏斗”上。这个过程耗时费力，且充满了不确定性。我们就像一个需求方，试图向一个能力强大但心智难以捉摸的第三方清晰地传达我们的意图，这其中的挑战远超预期。

实践中最大的感悟是，结构化是提升沟通效率的关键。通过类似README的格式，将Prompt拆解为角色（Role）、工具（Tools）、注意事项（Attention）、输出格式（Output）、执行逻辑（Logic）、具体要求（Requirements）等模块化指令，能够显著提高模型理解意图的准确率。这种方式的核心是尽可能地提供清晰、明确的上下文与背景信息，避免使用模糊的语言，从而帮助模型在庞大的可能性空间中，收敛到我们期望的输出范围。

即便采用了结构化的方法，我依然遇到许多“模型特色”的难题：

- “必须”并非绝对：指令中的“必须”、“一定”等强约束词，并不能保证100%被遵守。
- “重要”可能被忽略：模型有时会“抓不住重点”，无法准确识别我们强调的核心要素。
- 格式偶尔“叛逆”：即使明确规定了输出格式，模型有时仍会生成意料之外的“自由发挥”内容。

这些问题揭示了与大模型协作的本质：我们并非在编写具有确定性行为的代码，而是在通过自然语言引导一个概率系统。面对这种不确定性，需要更精细的策略，例如Few-short，即好的例子，清晰的逻辑链条和工具使用指南，迭代式优化等等。Prompt也不是越长越好，大模型的注意力可能会涣散。当然到Agent后考虑的不只是第一步的输入，而是上下文——所有输入给大模型的信息。

我还苦手于prompt工程，前期更多地追求可执行而不是质量。

GitHub - PandaBearLab/prompt-tutorial: chatGPT、prompt、LLM：https://github.com/PandaBearLab/prompt-tutorial

提示工程指南 — Nextra：https://www.promptingguide.ai/zh

设计的岔路口：三种 Agent 范式尝试

正是对这些底层逻辑的认知不足，才导致了我最初设计的失败。

- “全自主决策”Agent的阵亡

这是我最开始的尝试，也是最符合“Agentic”的方案。我给它设定了一个大的目标，提供了一篮子工具（克隆仓库、读写文件、执行 Shell 等），然后写下了一段充满信任和期望的 Prompt：

```
你是一个云计算/容器研发工程师。你的任务是为一个开源项目生成一个可部署到Kubernetes 1

## 你的最终目标:
创建一个与项目同名的、完整的Helm Chart，包含所有的yaml文件，符合helm的最佳实践。

## 注意事项:
    **优先找寻docker-compose文件**: ...
    **应用启动**: ...
    **依赖**: ...
    ...

## 你拥有的能力（工具）:
{tools}

## 你的工作原则与思考模式（非常重要）:
最优先原则：你应该像一个人类工程师一样，发现问题解决问题，不要停滞不前。请记住，你拥有
step1: ...
step2: ...
...
```

然后，我满怀期待地按下了回车。
Agent 确实“思考”了起来，它的 Thought 日志显示出一种努力工作的迹象。

- **决策瘫痪：** 在面对一个有多个 docker-compose-xxx.yml 文件的仓库时，它会陷入长久的思考：“我需要阅读xxx文件来明确部署方案”“我没有找到xxx文件”，这种思考让ai陷入了循环困境。
- **工具误用：** 它会幻想出一些不存在的文件路径，然后固执地调用 read_file_content 工具，在得到一连串“文件不存在”的 Observation 后，它会感到困惑。
- **幻觉频出：** 在深入分析一个复杂的 docker-compose 文件时，它可能会完全沉浸在理解某个服务的环境变量中，而多个文件可能存在的关联性让其产生“幻觉”。

- **无法掌控**：在多次运行测试时也会有那么一次执行成功，那一次Agent不仅完美定位到了文件，甚至部署文件中错误的路径地址也被识破。通过列出文件结构树，一一阅读依赖文件，Agent为一个有7个依赖的复杂项目成功生成了部署物。然而只是昙花一现，可以说再也没有复现成功，Agent好像忘了自己曾经有那么聪明过。这也引申出AI存在的一个问题，**每一次ReAct可能走上不一样的道路，而这可能是不可控的。**

阵亡反思：遵循了基本的ReAct行为范式，AI出现了**决策的脆弱性**： 在一个多步骤、长链条的任务中，任何一步的 Thought 出现偏差（比如错误地判断了文件的重要性），都可能导致整个任务链走向一个无法挽回的死胡同。Agent 很容易陷入“我该干啥”的循环，或者在一个细枝末节上“钻牛角尖”。

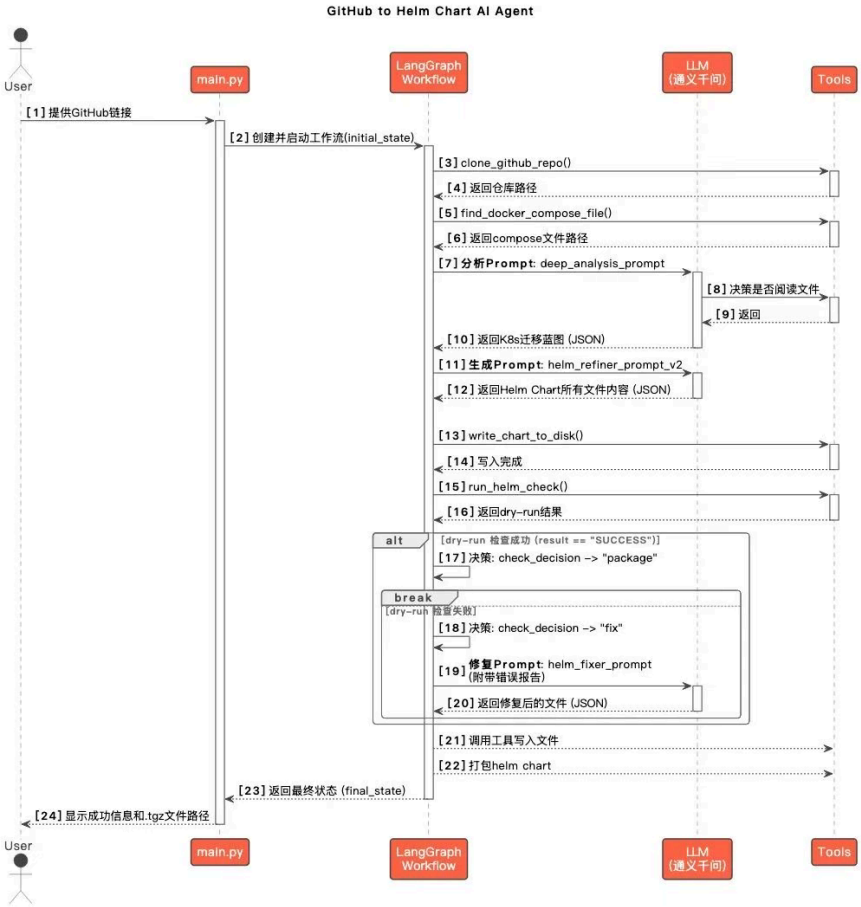
这次的阵亡，让我体会到理想与现实的差距。对于一个步骤清晰、逻辑严密的工程任务，当前 LLM 的**自主规划和纠错能力**还不够成熟 。如果只提供AI一个简单的prompt，然后把一个宏大的、定义模糊的任务完全交予 AI，就像让一个只有短期记忆的聪明人去完成一项需要长期规划的系统工程。**当然我觉得可能有很大的原因是prompt的锅，因为我既没有遵循提供给AI明确的流程的原则**，也没有一个好的Few-short。但由于调试prompt实在是容易陷入“调试地狱”，且是可能是一个需要长期试验的过程，而简单prompt的结果离预期太远，在初探阶段我放弃了。

- “结构化工作流”Agent

“全自主”的道路上行不通后，我换了一条道路。我意识到，**我需要的不是一个“思考者”，而是一个在我的指导下，能把活儿干得漂亮的“执行者”**。

于是，我放弃了让 AI 自由决策的幻想，转而设计了**一套固定的、结构化的工作流**。在这个范式里，**人类负责定义“骨架”（Workflow），AI 负责填充“血肉”（Analysis & Generation）**。并且为了快速获得测试结果，我将任务拆分，从一个项目的部署物生成到存在docker-compose项目的部署物生成。

我使用了 LangGraph 来编排这个工作流。下面是 MVP 的时序图：



最终输出：

输出

```
AI Agent工作流已构建。

==================================================
🚀 开始执行AI Agent任务...
==================================================

--- 节点: 拉取仓库 ---仓库已成功克隆到 ./workspace/WukongCRM-11.0-JAVA
-> 正在执行节点: find_compose_file
--- 节点: 寻找Compose文件 ---
找到Compose文件: ./workspace/WukongCRM-11.0-JAVA/docker/docker-compose.
--- 节点: 提取Compose文件中的本地文件引用 ---
提取到 6 个本地文件: ['workspace/WukongCRM-11.0-JAVA/docker/conf/mysql/m
-> 正在执行节点: extract_local_files
--- 节点: 读取本地文件内容 ---
-> 正在执行节点: read_local_files
--- 节点: 深度分析Compose文件 (迭代式生成) ---
识别到 7 个服务: ['db', 'redis', 'elasticsearch', 'nacos', 'xxl-job-admi
--- 正在为服务 'db' 生成蓝图... ---
✅ 成功生成并拼接服务 'db' 的蓝图。
```

```
--- 正在为服务 'redis' 生成蓝图... ---
✅ 成功生成并拼接服务 'redis' 的蓝图。
--- 正在为服务 'elasticsearch' 生成蓝图... ---
✅ 成功生成并拼接服务 'elasticsearch' 的蓝图。

...

🎉 所有服务的蓝图均已成功生成!
-> 正在执行节点: analyze_compose_file
--- 节点: 生成Helm Chart文件 (核心上下文迭代式生成) ---
识别到需要生成的 30 个Chart文件 (按顺序): ['Chart.yaml', 'values.yaml', 't
--- 正在为文件 'Chart.yaml' 生成内容... ---
✅ 成功生成文件 'Chart.yaml' 的内容。
--- 正在为文件 'values.yaml' 生成内容... ---
✅ 成功生成文件 'values.yaml' 的内容。
--- 正在为文件 'templates/_helpers.tpl' 生成内容... ---
✅ 成功生成文件 'templates/_helpers.tpl' 的内容。
--- 正在为文件 'templates/NOTES.txt' 生成内容... ---
✅ 成功生成文件 'templates/NOTES.txt' 的内容。
--- 正在为文件 'templates/dep-db-configmap-db-entrypoint-script.yaml' 生
✅ 成功生成文件 'templates/dep-db-configmap-db-entrypoint-script.yaml' 的

...

🎉 所有Chart文件均已成功生成!
-> 正在执行节点: generate_helm_chart_files
--- 节点: 将Chart文件写入磁盘 ---
--- 节点: Helm Lint检查 ---
对 './workspace/WukongCRM-11.0-JAVA/WukongCRM-11.0-JAVA' 执行 helm lint
Helm lint 失败:
==> Linting ./workspace/WukongCRM-11.0-JAVA/WukongCRM-11.0-JAVA
[ERROR] Chart.yaml: unable to parse YAML
  error converting YAML to JSON: yaml: line 42: found character that c
[ERROR] templates/: cannot load Chart.yaml: error converting YAML to J
[ERROR] : unable to load chart
  cannot load Chart.yaml: error converting YAML to JSON: yaml: line 42

Error: 1 chart(s) linted, 1 chart(s) failed
⚠️ Lint检查失败 (尝试次数 1/50), 进入修复流程。
-> 正在执行节点: helm_lint
--- 节点: 修复Chart文件 ---
调用通义千问修复Chart文件...
LLM提出了 1 个文件的修复方案。
--- 节点: Helm Lint检查 ---
对 './workspace/WukongCRM-11.0-JAVA/WukongCRM-11.0-JAVA' 执行 helm lint
Helm lint 失败:
```

```
==> Linting ./workspace/WukongCRM-11.0-JAVA/WukongCRM-11.0-JAVA
[ERROR] templates/: template: wukongcrm-11-0-java/templates/dep-elast

Error: 1 chart(s) linted, 1 chart(s) failed
⚠️ Lint检查失败（尝试次数 2/50），进入修复流程。
-> 正在执行节点: helm_lint
--- 节点: 修复Chart文件 ---
调用通义千问修复Chart文件...
LLM提出了 2 个文件的修复方案。
-> 正在执行节点: fix_chart
--- 节点: Helm Lint检查 ---
对 './workspace/WukongCRM-11.0-JAVA/WukongCRM-11.0-JAVA' 执行 helm lint
Helm lint 通过!
✅ Lint检查通过，流程继续到打包步骤。
-> 正在执行节点: helm_lint
--- 节点: Helm Install Dry-Run 检查 ---
✅ helm install --dry-run 检查通过。
✅ Install Dry-Run 检查通过，流程继续到打包步骤。
--- 节点: 将Chart文件写入磁盘 ---
-> 正在执行节点: write_chart_to_disk
-> 正在执行节点: helm_install_check
--- 节点: 打包Helm Chart ---
Chart成功打包到: ./workspace/WukongCRM-11.0-JAVA/wukongcrm-11-0-java-0.
-> 正在执行节点: package_chart

==================================================
✅ AI Agent任务执行完毕!
==================================================

🎉 任务成功!
生成的Helm Chart包位于: ./workspace/WukongCRM-11.0-JAVA/wukongcrm-11-0-

你可以使用以下命令来部署它:
helm install my-release ./workspace/WukongCRM-11.0-JAVA/wukongcrm-11-0
```

中间语言“部署蓝图”

由于上一个“全自主决策”Agent的失败，我发现可能更需要流程的可控性。如果让AI自己生成部署蓝图，后续再遵循蓝图进行部署物生成，可能会让结果更为可控。

我引入了一个中间步骤：让 AI 先把项目的docker-compose内容**翻译**成一份详尽的、结构化的**“部署蓝图”JSON**。

Prompt 长这样：

```
**角色**: 你是一位顶级的云原生解决方案架构师...
**任务**: 请对以下 `docker-compose.yml` 文件进行深度分析，并生成一份详尽的、结构
**`docker-compose.yml` 内容**:
```yaml
{docker_compose_content}
相关联的本地文件内容:
{related_files_content}
核心分析指令:
1. 识别主应用: ...
2. 依赖关系与启动顺序: ...规划使用Init Containers...
3. 挂载（Volumes）意图分析: ...映射到PersistentVolumeClaim...ConfigMap...S
...
输出要求:
你必须严格按照以下JSON格式输出你的蓝图...
{{
  "main_application": {{ ... }},
  "dependencies": [ {{ ... }} ]
}}
```

为什么设计这个“蓝图”？

1. 降低认知负荷，保证分析质量：让 AI 专注于“理解和分析”，而不是同时分心去想 Helm 的模板语法。这极大地提升了分析阶段的准确性。AI 在这一步表现得像一个真正的架构师，它能准确识别数据卷、配置文件、环境变量，并规划出对应的 K8s 资源。

2. 应对 Token 限制：复杂项目部署物的输出可能会超过模型的上限。为此采用了“迭代分片法”，它会先解析出里的所有服务，然后一个一个地调用 LLM 去生成每个 service 的蓝图片段，最后在代码层面将它们拼接成完整的蓝图。用工程手段弥补模型能力限制

3. 解耦与稳定：“蓝图”成了一个稳定、可调试的中间状态。如果最终生成的 Chart 有问题，我能清晰地判断，是“蓝图”生成错了，还是从“蓝图”到“YAML”的翻译步骤错了。这种解耦，对于调试和迭代至关重要。看似是使用了更多的prompt，但减少了调试的难度。

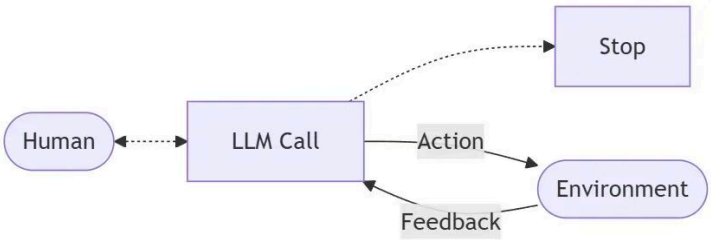
自我修复的“自愈循环”

“一次就写对所有代码”是神话，AI 也不例外。它生成的 Helm Chart 会有各种语法错误、命名不规范、模板引用错误等问题。设计一个**自愈循环**，并将其拆开单独形成一个工作逻辑，能够隔离开生成和修复环节，尽量避免大模型陷入无限循环，也防止出现“越修越错”。
```

把dry run这个确定性的、高质量的外部反馈，整合进了 AI 的工作流中。Agent 不再是“盲目生成”，而是有了“试错和修正”的能力 。在实践中，这个循环可能在20次内修复大部分常见的 Chart 问题，极大地提升了最终交付物的“可用性”。

这个“结构化工作流”方案，看起来是缺少了Agentic，但它僵硬而准确，最终成功地支撑起了我的 MVP。它让我明白，在当前的 AI 技术水平下，**好的 Agent 应用，是工程设计与 AI 能力的精妙结合，而不是对 AI 的盲目放权。**

这个部分正好贴合了Antropic的 Barry Zhang 提出 Agent 的概念，即在循环（Loop）中使用工具的模型。



（图源：https://generativeai.pub/use-n8n-like-a-pro-with-anthropics-6-simple-composable-patterns-for-llms-d601efe0f314）

- “多 Agent 协作”

单一 Agent 完成一个端到端的复杂任务，它既要懂宏观架构，又要懂代码细节，还要会调试。在复盘 MVP 时，我觉得一个复杂任务，应该由一个 Agent 团队来协作完成 。

- 总指挥(Orchestrator): 接收原始需求，进行任务分解。
- Agent 1: 分析专家(Analysis Agent):
  - 输入: GitHub URL。
  - 任务: 深入分析项目，阅读代码、README、甚至模拟人类去搜索相关文档，最终产出项目的核心元数据和一份或多种“可行的部署方案”（比如“方案A：用 Docker Compose 部署”，“方案B：用源码编译”）。
  - 输出: 结构化的部署方案 JSON。
- Agent 2: 执行专家群(Execution Agents):
  - Agent 2a: Docker Compose 执行器: 这就是我们 MVP 的核心。输入部署方案A，输出 Helm Chart。
  - Agent 2b: 源码编译执行器: 输入部署方案B，负责生成包含编译步骤的 Dockerfile 和对应的 K8s YAML。
  - Agent 2c: ...
- Agent 3: 质检专家(QA Agent):

- 输入：生成的部署产物（如 Helm Chart）。
- 任务：进行静态检查（lint）、动态部署验证（在沙箱 K8s 环境中 helm install），并报告成功或失败。
- 输出：质检报告。

这个“多 Agent 协作”模式，`分析专家` 就是 Planner，它负责思考和规划；`执行专家群` 就是 Workers，它们各自拥有一套专用工具，负责高效地执行具体的子任务。

这种模式的好处是显而易见的：

- 单一职责： 每个 Agent 的目标更明确，Prompt 和工具集可以高度优化 。
- 可扩展性： 增加一种新的部署方式，只需要增加一个新的“执行专家”Agent，而不用修改整个系统。
- 健壮性： 单个 Agent 的失败不会导致整个系统崩溃，总指挥可以进行重试或切换到备用方案。

但是这也引入了新的问题，那就是多agent框架是怎样的，规范好用的多agent的传输协议，上下文和memory的设计等等，这又是一个很大的探索话题。

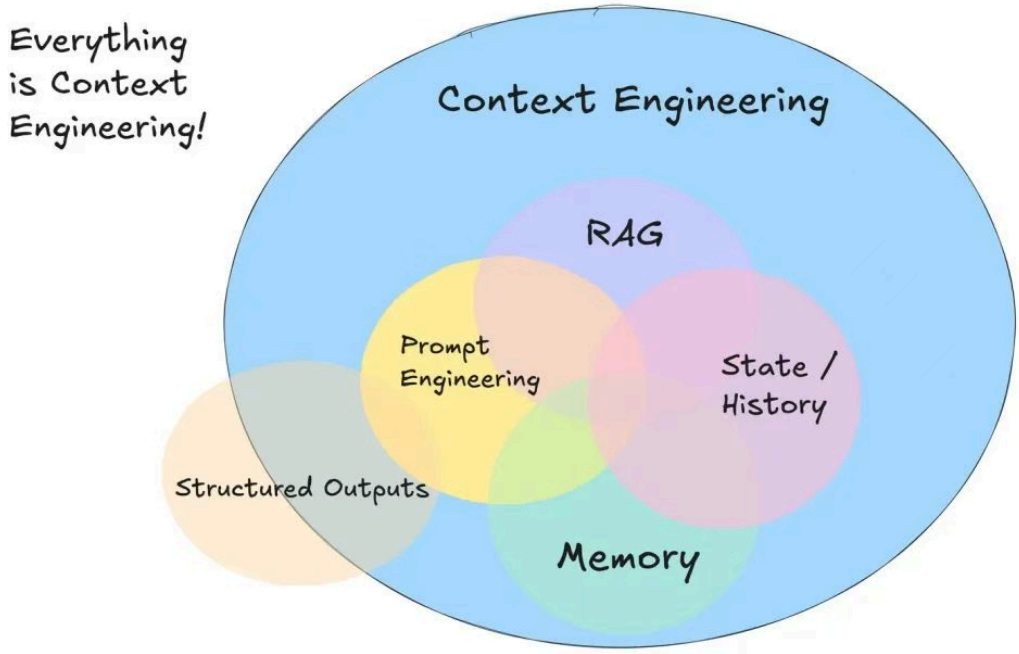
03 一个好的 Agent 到底长什么样？

大模型是不可控的。不是‘给LLM一堆工具让它自由发挥’，而是大部分由确定性代码构成，在关键决策点巧妙地融入LLM能力。

12-Factor Agents

初探让我对“一个好的 Agent 到底长啥样？”这个问题，有了更深刻的理解。12-Factor Agents：https://github.com/humanlayer/12-factor-agents?tab=readme-ov-file 这个项目为构建企业级、生产级的 Agent 提供了极好的指导原则。结合它的思想和我的实践，有一些感悟：

1. 结构化上下文：上下文包括Prompt、说明、RAG 文档、历史记录、工具调用、记忆。这是做好一个agent大概率的选择，结构化的输入能让大模型更好地理解任务、执行任务，而结构化的输出也是为了下一步的输入。



2.单一职责： 这是我从“全自主 Agent”失败到“多 Agent 协作”思考的最大转变。试图构建一个无所不能的“瑞士军刀”式 Agent，可能导致样样通、样样松。而将任务拆解，让每个 Agent 专注于一个明确的、有边界的任务，是通往成功的首要法则 。

3.外部化状态管理： Agent 的思考过程应该是无状态的。所有的状态，都应该由外部的工作流引擎来管理 。这使得流程的每一步都可以被独立地调试、重试和观察。

4.解耦处理阶段： “蓝图”设计，就是这个原则的体现。将“分析理解”、“代码生成”、“检查修复”等阶段清晰地解耦，使得每个阶段都可以被独立优化，也让整个流程的复杂性得到了有效控制 。

5.可控性与可预测性： 这是 12-Factor 之外，个人的感悟。在工程领域的某些场景下，我们追求的往往不是 AI 的惊艳，而是它的可靠。一个好的 Agent，其行为应该是高度可控和可预测的。“结构化工作流”就是一种增强可控性的实践。我们应该通过清晰的流程、明确的指令和严格的输出格式。如何平衡AI的创造性和确定性是需要在一开始就确定的。

04 实践中的痛点

AI 可观测

市面上有很多ai可观测产品，这次使用的是LangSmith。LangSmith是LangChain团队推出的一个统一的可观测性与评估平台，旨在帮助开发者对AI应用进行调试、测试和监控。鉴于大

型语言模型（LLM）本质上的非确定性，其内部运作机制难以完全理解，这给应用开发带来了独特的调试挑战。LangSmith通过提供LLM原生的可观测性能力，允许开发者分析调用链路（Traces），监控关键指标，从而追踪错误并优化应用性能。

这样的工具非常强大，它能让我清晰地看到 AI 的每一次思考、每一次工具调用 。但是缺少根因定位，比如能够分析错误链路（如工具调用失败→Token超限→LLM超时）。可能大模型调用发生错误或者停止了，但LangSmith 里的 AI Trace 一片祥和，所有步骤都是绿色的，而我可能需要猜测这一次是不是token超限了。

另外一个想法，是不是可以将 AI Trace 与业务观测体系融合，实现真正的端到端、一站式可观测。对于一个开源ai可观测来说这可能不是必须的，但对于应用管控来说我觉得是可以考虑的。

维度	LangSmith	Prometheus	ARMS
定位	AI应用可观测性（LLM链式调用、提示工程、工具交互）	基础设施监控（服务器、容器、中间件指标）	全链路应用性能管理(APM)（分布式追踪、JVM分析）
核心数据	LLM输入/输出、工具调用链、提示词版本、Token消耗	时间序列指标（CPU/内存/网络）	代码级调用栈、SQL性能、异常堆栈
技术特性	- 基于LangChain深度集成- 支持Prompt动态调试- AI辅助评估	- 多维数据模型+PromQL- 拉取(Pull)模式采集	- 无缝整合Arthas诊断- 方法级实时追踪
典型场景	调试Agent决策路径、优化RAG召回效果	服务器资源预警、K8s集群监控	定位Java应用性能瓶颈、SQL慢查询根因

Prompt 工程的“炼丹术”困境

如果说 Agent 的代码是骨架，那 Prompt 就是灵魂。但这个“灵魂”的注入过程，带着一点玄学。

最佳实践的缺失与高昂的试错成本： OpenAI 的最佳实践：https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api给了很多好的指导，比如提供清晰指令、给出示例等。但具体到我的场景，“什么样的结构才是最省 Token 且信息量最高的？”“Few-shot 示例应该怎么说明？”这些问题没有标准答案，全靠一次次尝试 。而每一次尝试，都是时间、精力、token的消耗。

版本管理的噩梦： 我曾经为了修复一个因为 env_file 路径解析错误导致的 Bad Case，精心修改了prompt。结果，这个 Case 跑通了，但另外五个原本正常的 Case 全都挂了，AI 开始在其他地方产生幻觉。这种“牵一发而动全身”的效应，让 Prompt 的迭代和回归测试变得异常困难。想要一个能够清晰进行对比每一次输出结果的prompt调试工具，科学的版本管理和 A/B 测试框架。

可解释性的黑洞： 我改了 Prompt 里的一个词，Agent 的行为就变了。但为什么？是哪个词、哪个句子、哪个示例在起作用？它的权重是多少？这种解释性的缺失，让我们在调优时，更像是在“炼丹”，充满了偶然性和不确定性。

AI的“不确定性”

我把模型的 `temperature` 设为 `0`，希望能获得确定性的输出 。但在复杂的推理任务中，即使 `temperature` 为 `0`，模型在不同时间点对完全相同的输入，也可能给出细微差别甚至完全不同的推理路径。在需要精确输出的工程场景下，这种不确定性是致命的。我们该如何设计系统，来拥抱 AI 在“分析、发散”阶段的“有益的不确定性”，同时又能在“生成、收敛”阶段约束其行为，获得“可靠的确定性”？这或许是未来 Agent 需要解决的核心哲学命题。



这次探索，除了在 Agent上的思考，还有一些需要复盘的东西。

- 一个项目拿到手，要先做设计，切忌东一榔头西一榔头。

在面对 AI 这种充满不确定性的新技术时，被“先跑起来看看”的冲动所支配。急于看到 AI 调用的神奇效果，却忽略了对整个工作流的顶层设计。结果就是，AI 的行为像一个无头苍蝇，修补工作也变得支离破碎。越是面对不确定的技术，就越需要一个确定性的工程设计作为锚点。

- 关于学会做减法，也学会做加法。

做减法指的是分解复杂问题，简化方案，从最小可行方案开始。当试图构建一个无所不能、能处理所有异常的“超级 Agent”时，很快就发现这是一个非常困难的任务。这时就要做减法：承认 AI 的局限性，将大的目标分解。这是在资源和时间有限的情况下的选择。

做加法指的是打破工具和方法的门户之见，用一种开放和务实的心态，组合一切可用的资源。最终的目标只有一个：让问题被解决，让项目能落地。



这个探索其实才刚刚开始，现在的认知可能是片面的，尝试也可能是不到位的。再加上AI相关技术的快速发展，也许接下来会推翻今天的一些想法，但是目标是确定的——解决工程问题，让项目最终可靠地落地。不管是什么样的方法或是范式，它们都只是工具箱中的选项。我们的任务，就是在理解这些工具的边界和特性的基础上，遇到问题，分析问题，并用最恰当的方式去解决问题。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 7051